

Assignment 3: Reproducible Sequence Analysis Pipeline

June 12, 2026

1 Introduction

In bioinformatics, sequence comparison is one of the most common computational tasks. Researchers often need to compare DNA sequences against a reference sequence, compute similarities, generate alignments, and summarize the results in a reproducible and automated way.

In this assignment, you will develop a small bioinformatics workflow for analyzing DNA sequences. Starting from a FASTA file, you will extract sequence information, compare sequences using Dynamic Programming, generate alignment summaries, analyze the results using Regular Expressions, and automate the complete workflow using modern reproducibility tools.

Throughout the assignment, you will apply several concepts studied during the course:

- Dynamic Programming
- Regular Expressions
- Conda Environments
- Workflow Automation with Snakemake

You are provided with the FASTA file `PM_50.fasta`, which contains 50 DNA sequences from *Papilio machaon*. The objective is to construct a reproducible analysis pipeline capable of generating all requested outputs directly from this input file.

General Requirements

- All programs must be written in Python 3.
- External bioinformatics libraries (e.g., Biopython) are not allowed unless explicitly stated.
- All scripts must be executable from the command line.
- Input and output file names must match those specified in the assignment.
- The workflow must execute correctly using Snakemake.

- Include comments explaining the most important parts of your code.
- All output files must be generated automatically from the provided FASTA file and they must be tab-separated (TSV format).

Part 1: FASTA Parsing and Regular Expressions (1.5 points)

In this part, you will extract information from a FASTA file using Python and Regular Expressions.

The FASTA file contains a collection of DNA sequences. Each sequence begins with a header line starting with the character `>` followed by an accession identifier and additional descriptive information.

For example:

```
> NW_014478584.1 Papilio machaon isolate ...
```

The value immediately following the `>` symbol is called the **accession identifier**. This identifier uniquely identifies a sequence within a biological database. The sequences in the FASTA file may span multiple lines. Therefore, your program must correctly reconstruct the complete sequence before computing its length.

Using the information provided above, implement a Python program named `parse_fasta.py` that reads the FASTA file and extracts, for each sequence:

- The accession identifier.
- The total sequence length.

The accession identifier must be extracted using a Regular Expression. For example, from the header:

```
> NW_014478584.1 Papilio machaon isolate ...
```

your program should extract:

```
NW_014478584.1
```

Furthermore, the program must generate an output file named `sequences.tsv` containing one row per sequence with the following format:

Accession	Length
NW_014478584.1	1043
NW_014478585.1	892
...	

Part 2: Dynamic Programming Edit Distance (3.5 points)

In this part, you will compare DNA sequence fragments using the edit distance algorithm implemented with Dynamic Programming.

The **edit distance** between two sequences is the minimum number of edit operations required to transform one sequence into the other. In this assignment, the allowed edit operations are:

- **Substitution:** replacing one nucleotide by another.
- **Insertion:** inserting a nucleotide.
- **Deletion:** deleting a nucleotide.

Each operation has a cost of 1. A match between equal nucleotides has cost 0.

Use the first 18 nucleotides of the first sequence in `PM_50.fasta` as the reference sequence. This reference sequence must be extracted automatically from the FASTA file and must not be hard-coded in the program. Then, for every sequence in the FASTA file, extract its first 18 nucleotides and compute the edit distance between this fragment and the reference sequence.

This restriction to 18 nucleotides is intentional. It keeps the Dynamic Programming matrices small enough to inspect manually, while still allowing you to apply the same algorithm used for larger sequence alignment problems.

Implement a function named `edit_distance_dp(pattern, text)` where `pattern` is the reference sequence and `text` is the sequence fragment being compared.

Your implementation must use a **bottom-up tabulation approach**. This means that you must build a Dynamic Programming matrix where each cell stores the solution to a smaller subproblem.

If the reference sequence has length n and the sequence fragment has length m , the DP matrix must have size: $(n + 1) \times (m + 1)$

The cell `M(i, j)` must represent the edit distance between:

`pattern[0:i]` and `text[0:j]`

That is, between the first i characters of the reference sequence and the first j characters of the sequence fragment.

The first row and the first column must be initialized to represent alignments against the empty string. For example, transforming a sequence of length i into an empty sequence requires i deletions.

For each remaining cell, compute the minimum cost among the three possible ways of reaching that cell:

- Diagonal movement: match or substitution.
- Upward movement: deletion.
- Left movement: insertion.

The final edit distance must be obtained from the bottom-right cell of the matrix.

Your implementation must satisfy the following requirements:

- Build the complete DP matrix.
- Compute the edit distance using the matrix.
- Do not use external libraries to compute the edit distance.
- The algorithm must work for any pair of input strings, not only for the provided FASTA file.

For every sequence in `PM_50.fasta`, generate a file named `distances.tsv` with the following format:

Accession	Distance
NW_014478584.1	0
NW_014478585.1	5
...	

The first sequence should have distance 0 because its first 18 nucleotides are used as the reference sequence.

In addition, for the first sequence in the FASTA file, print the complete Dynamic Programming matrix and include it in `answers.pdf`. You must also briefly explain:

- What each cell $M(i, j)$ represents.
- Why Dynamic Programming avoids the repeated computations that appear in a brute-force recursive solution.
- Why the time complexity of this tabulation algorithm is $O(n \cdot m)$.

Part 3: CIGAR Generation (2 points)

In this part, you will extend your edit distance implementation to recover not only the final distance, but also the sequence of operations that produced the optimal alignment. The Dynamic Programming matrix gives the minimum edit distance between two sequences. However, the final value alone does not tell us which operations were used to transform one sequence into the other. To recover this information, you must perform a **traceback** through the matrix, starting from the bottom-right cell and moving backwards until reaching the top-left cell.

During the traceback, each movement corresponds to one alignment operation:

- A diagonal movement with equal characters corresponds to a **M** operation, meaning **Match**.
- A diagonal movement with different characters corresponds to a **X** operation, meaning **Substitution**.
- A movement to the left corresponds to an **I** operation, meaning **Insertion**.
- A movement upwards corresponds to a **D** operation, meaning **Deletion**.

In this assignment, you must use the **expanded CIGAR representation**. This means that each operation must be written explicitly. Extend your Python program so that, for each sequence fragment compared against the reference, it reports both the edit distance and the corresponding expanded CIGAR string. Generate an output file named `alignments.tsv` with the following format:

Accession	Distance	CIGAR
NW_014478584.1	3	MMXMMIIDM
NW_014478585.1	5	MMXXMDIIMM
...		

The file must contain one row per sequence in the FASTA file. The **Distance** column must correspond to the final edit distance computed from the Dynamic Programming matrix, and the **CIGAR** column must correspond to one optimal traceback path.

If multiple optimal traceback paths are possible, any one of them will be accepted, as long as it is consistent with the reported edit distance.

Part 4: Regular Expression Analysis of CIGAR Strings (1 point)

In this part, you will use Regular Expressions to analyze the CIGAR strings generated in Part 3.

The CIGAR string summarizes the sequence of edit operations required to transform the reference sequence into a target sequence. Each character in the CIGAR string represents a specific alignment operation:

- M: Match
- X: Substitution
- I: Insertion
- D: Deletion

For example, the CIGAR string `MMXMMIIDM` indicates that the optimal alignment contains matches, substitutions, insertions and deletions.

The goal of this part is to extract useful information from these CIGAR strings using the Regular Expression techniques studied during the course.

For each sequence, analyze the corresponding CIGAR string and determine:

- The total number of Matches (M)
- The total number of Insertions (I)
- The total number of Deletions (D)
- The total number of Substitutions (X)

You must use Regular Expressions to perform this analysis. Examples of useful functions include:

```
re.findall()
re.finditer()
```

Generate an output file named `cigar_stats.tsv` with the following format:

Accession	M	I	D	X
NW_014478584.1	12	2	1	3
NW_014478585.1	10	4	0	4
...				

Each row must contain the statistics computed from the corresponding CIGAR string. Using the information stored in both `distances.tsv` and `cigar_stats.tsv`, generate a summary report named `report.txt`. The report must identify:

- The sequence with the smallest edit distance.
- The sequence with the largest edit distance.
- The sequence with the largest number of matches.

If multiple sequences satisfy the same criterion, report any one of them. For each case, include:

- The accession identifier.
- The corresponding edit distance.
- The associated CIGAR statistics.

An example report could look like:

```
Sequence with smallest edit distance:
NW_014478584.1
Distance: 2
Matches: 15
Insertions: 1
Deletions: 0
Substitutions: 1
```

```
Sequence with largest edit distance:
NW_014478590.1
Distance: 8
Matches: 9
Insertions: 3
Deletions: 2
Substitutions: 3
```

Briefly explain in `answers.pdf` why Regular Expressions are useful for extracting information from CIGAR strings and other bioinformatics file formats.

Part 5: Conda Environment (1 point)

One of the main challenges in computational research is ensuring that an analysis can be reproduced on different computers and at different points in time. Different versions of Python and software packages can lead to inconsistent results or execution failures. To address this problem, Conda allows users to create isolated software environments containing all the packages required to execute a workflow. By sharing an environment description file, other users can recreate exactly the same software configuration.

In this part, you will create a Conda environment containing all software required to execute the workflow developed in this assignment.

Create a Conda environment that includes the packages necessary to run:

- The FASTA parsing program.
- The Dynamic Programming alignment program.
- The CIGAR analysis program.
- The Snakemake workflow.

After creating and testing the environment, export its configuration to a file named `environment.yml`

This file should contain all package names and versions required to reproduce your software environment.

Include the generated `environment.yml` file in your submission.

In addition, answer the following questions in `answers.pdf`:

1. Which command creates the environment?
2. Which command activates the environment?
3. Which command exports the environment?
4. Why is an environment file useful for reproducibility?

Finally, briefly explain:

- Which packages were included in your environment.
- Why these packages are required for the workflow.
- How another user could recreate your environment using the exported `environment.yml` file.

Part 6: Workflow Automation with Snakemake (1 point)

In this part, you will automate the complete analysis using Snakemake.

Until now, the workflow could be executed manually by running each Python script in the correct order. However, this approach is error-prone because users must remember which script depends on which input files and which output files must be generated first.

Snakemake solves this problem by describing the workflow as a set of rules. Each rule specifies:

- The input files required by a step.
- The output files generated by that step.
- The command needed to produce the output files from the input files.

Create a file named **Snakefile** that automates the complete sequence analysis pipeline. The workflow must represent the following dependency structure:

```
PM_50.fasta
  |
  v
parse_fasta.py
  |
  v
sequences.tsv
  |
  v
alignment.py
  |
  v
alignments.tsv
  |
  v
cigar_stats.py
  |
  v
cigar_stats.tsv
  |
  v
report.txt
```

Your Snakefile must contain at least the following rules:

- **rule all:** defines the final target of the workflow. The final target must be `report.txt`.
- **rule parse:** executes `parse_fasta.py` and generates `sequences.tsv`.

- **rule align:** executes `alignment.py` and generates `alignments.tsv`.
- **rule stats:** executes `cigar_stats.py` and generates both `cigar_stats.tsv` and `report.txt`.

The rule names shown above are suggestions. Equivalent rule names are acceptable provided the workflow structure remains unchanged.

Each rule must correctly define its `input`, `output`, and `shell` sections. The workflow should be executable from the command line using:

```
snakemake --cores 1
```

Before running the complete workflow, test it using a dry run:

```
snakemake -n
```

The dry run should show which rules Snakemake plans to execute, without actually generating the output files.

You must also generate the Directed Acyclic Graph (DAG) of the workflow using:

```
snakemake --dag | dot -Tpng > dag.png
```

Include the generated `dag.png` file in your submission.

In `answers.pdf`, include:

- The output of `snakemake -n`.
- The generated workflow DAG.
- A brief explanation of what each rule does.
- A short explanation of why Snakemake improves reproducibility compared with manually running each script.

2 Deliverables

Submit a compressed file named `assignment3_lastname_firstname.zip` containing the following directory structure:

```
assignment3/
|
|-- PM_50.fasta
|-- parse_fasta.py
|-- alignment.py
|-- cigar_stats.py
|-- Snakefile
|-- environment.yml
|-- sequences.tsv
|-- distances.tsv
|-- alignments.tsv
|-- cigar_stats.tsv
|-- report.txt
|-- dag.png
'-- answers.pdf
```

The generated files should correspond to the outputs obtained from running your workflow on the provided FASTA file.

The file `answers.pdf` must contain:

- The complete Dynamic Programming matrix for the first sequence.
- Answers to all conceptual questions included in the assignment.
- The output of `snakemake -n`.
- A brief explanation of the workflow rules.
- A short discussion of the reproducibility benefits of Conda and Snakemake.

Submit the compressed file through the Virtual Campus before the indicated deadline.

Important:

- All Python scripts must execute correctly.
- The workflow must be executable through Snakemake.
- The `Snakefile` must be capable of regenerating all output files from the original `PM_50.fasta` input file.
- The submitted `environment.yml` file must contain all software dependencies required to execute the workflow.